Laboratorul 6. Informații invizibile: forensics și steganografie

fișă de laborator · 2 × 90 min (cu pauză la mijloc) · două lucrări · se lucrează individual, la terminal

CE CONSTRUIȚI ASTĂZI. Două unelte de forensics, scrise de voi în Python, care intră în arsenal. Lucrarea A este un decoder universal: dat un șir, recunoaște stratul (base64, hex, URL), îl desface, iar dacă e un XOR cu cheie scurtă sau un hash de parolă, îl sparge prin forță brută sau dicționar. Lucrarea B este forensics pe fișiere: citești semnătura reală, scoți textul și metadatele (inclusiv GPS) dintr-o poză, extragi un fișier ascuns, apoi ascunzi și dezvălui un mesaj prin steganografie LSB. Amândouă se predau cum cere regula de fier: cod și output, niciodată o captură de ecran.

ȚINTE PERMISE ȘI CADRUL LEGAL. Lucrați DOAR pe fișierele pregătite pentru laborator (le generați voi la pasul de pregătire) și pe propriile voastre fișiere. Forensics pe dispozitivul, contul sau fișierele altcuiva, fără acordul lui, este acces neautorizat la date, o infracțiune (Cod penal RM, capitolul XI). Aceleași unelte, pe ce vă aparține, sunt muncă legală. Diferența nu este unealta, este permisiunea.

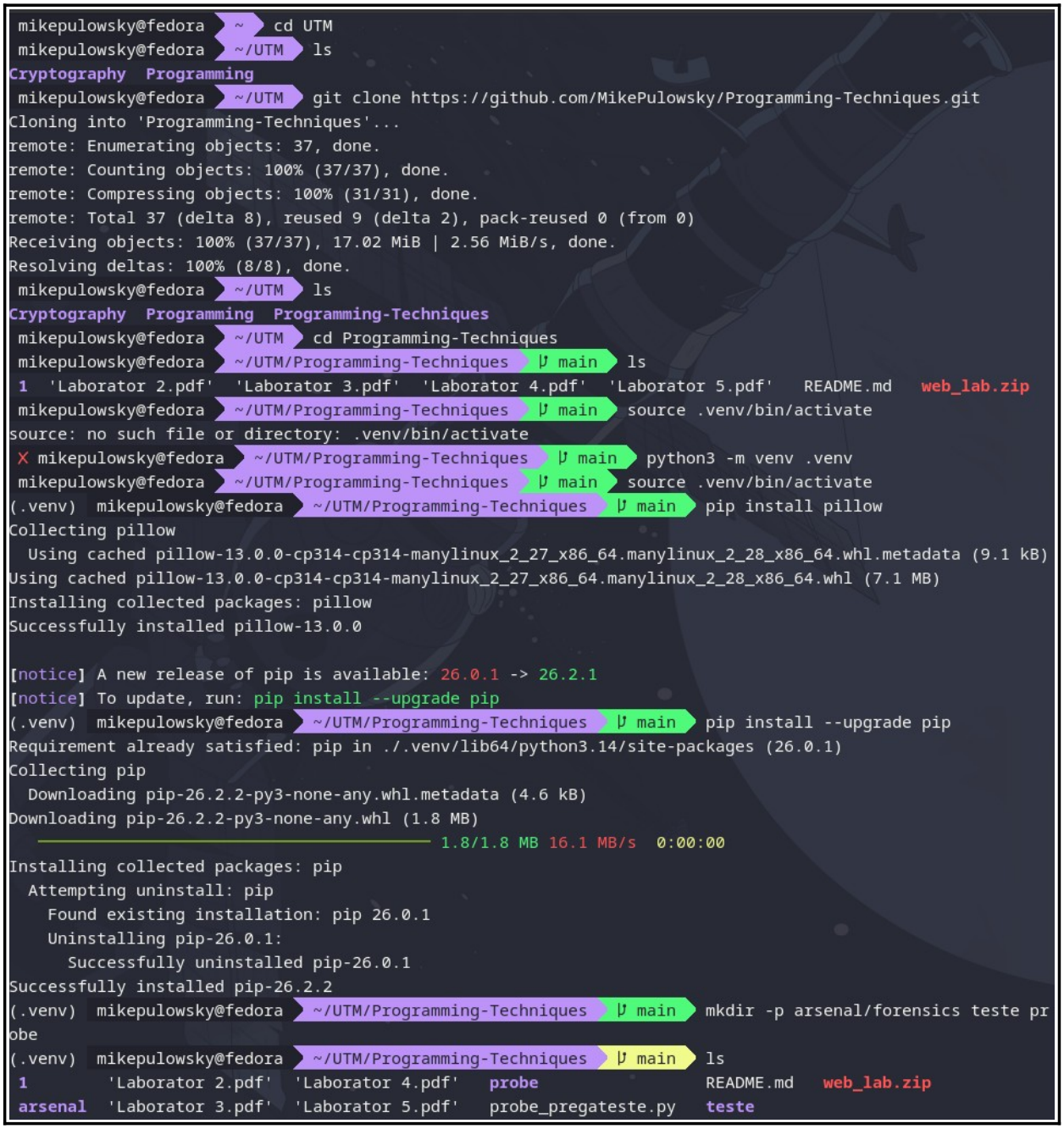
Pregătire (comună celor două lucrări)

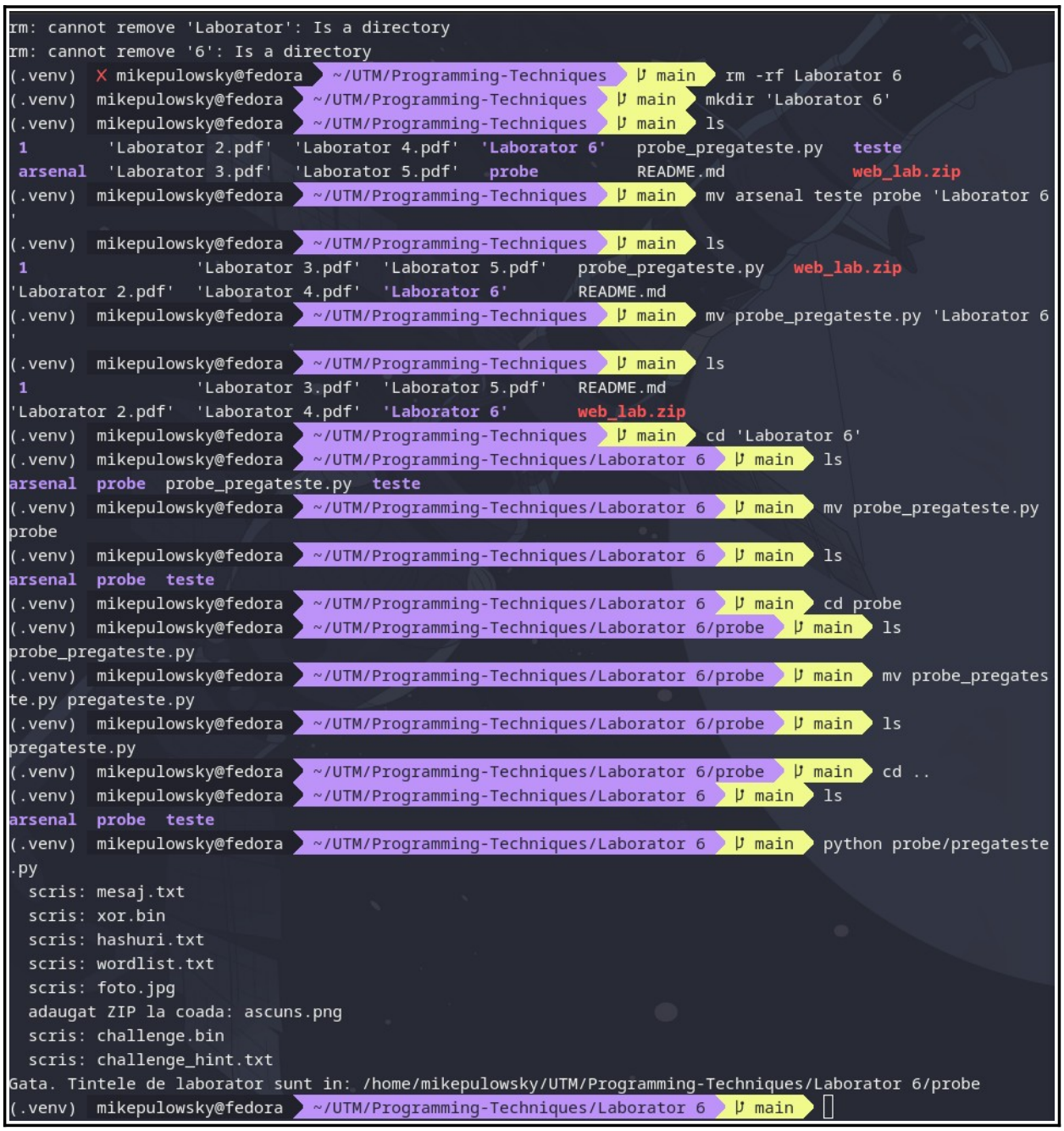
Intră în depozitul disciplinei, activează mediul virtual și instalează biblioteca de imagini. Apoi rulează scriptul de pregătire de mai jos: el îți creează, chiar la tine, fișierele pe care le vei analiza (un șir codat în straturi, un mesaj XOR, o poză cu EXIF și GPS, și o poză cu un fișier ascuns). Astfel toată lumea lucrează pe aceleași ținte, repetabil.

```
cd tehnici-prog-<nume>
# activeaza mediul (Windows PowerShell):
.\.venv\Scripts\Activate.ps1
# sau Linux: source .venv/bin/activate
pip install pillow
mkdir -p arsenal/forensics teste probe
# creeaza fisierele de laborator:
python probe/pregateste.py   # scrie in probe/
```

Scriptul probe/pregateste.py îți este dat (în depozitul de start al laboratorului). El produce: probe/mesaj.txt (șir în mai multe straturi de codare), probe/xor.bin (text criptat cu XOR pe un octet), probe/hashuri.txt (câteva hash-uri de spart), probe/foto.jpg (cu EXIF și GPS) și probe/ascuns.png (cu o arhivă lipită la coadă). Dacă lipsește, cere-l asistentului.

> **Ce poate crăpa.** Dacă import PIL dă eroare, biblioteca se numește la instalare pillow (nu „PIL"). Dacă nu vezi (.venv) în prompt, mediul nu e activat.


mikepulowsky@fedora ~ cd UTM
mikepulowsky@fedora ~/UTM ls
Cryptography Programming
mikepulowsky@fedora ~/UTM git clone https://github.com/MikePulowsky/Programming-Techniques.git
Cloning into 'Programming-Techniques'...
remote: Enumerating objects: 37, done.
remote: Counting objects: 100% (37/37), done.
remote: Compressing objects: 100% (31/31), done.
remote: Total 37 (delta 8), reused 9 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (37/37), 17.02 MiB | 2.56 MiB/s, done.
Resolving deltas: 100% (8/8), done.
mikepulowsky@fedora ~/UTM ls
Cryptography Programming Programming-Techniques
mikepulowsky@fedora ~/UTM cd Programming-Techniques
mikepulowsky@fedora ~/UTM/Programming-Techniques main ls
1 'Laborator 2.pdf' 'Laborator 3.pdf' 'Laborator 4.pdf' 'Laborator 5.pdf' README.md web_lab.zip
mikepulowsky@fedora ~/UTM/Programming-Techniques main source .venv/bin/activate
source: no such file or directory: .venv/bin/activate
X mikepulowsky@fedora ~/UTM/Programming-Techniques main python3 -m venv .venv
mikepulowsky@fedora ~/UTM/Programming-Techniques main source .venv/bin/activate
(.venv) mikepulowsky@fedora ~/UTM/Programming-Techniques main pip install pillow
Collecting pillow
Using cached pillow-13.0.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.1 kB)
Using cached pillow-13.0.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (7.1 MB)
Installing collected packages: pillow
Successfully installed pillow-13.0.0
[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
(.venv) mikepulowsky@fedora ~/UTM/Programming-Techniques main pip install --upgrade pip
Requirement already satisfied: pip in ./.venv/lib64/python3.14/site-packages (26.0.1)
Collecting pip
Downloading pip-26.2.2-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.2.2-py3-none-any.whl (1.8 MB)
1.8/1.8 MB 16.1 MB/s 0:00:00
Installing collected packages: pip
Attempting uninstall: pip
Found existing installation: pip 26.0.1
Uninstalling pip-26.0.1:
Successfully uninstalled pip-26.0.1
Successfully installed pip-26.2.2
(.venv) mikepulowsky@fedora ~/UTM/Programming-Techniques main mkdir -p arsenal/forensics teste probe
(.venv) mikepulowsky@fedora ~/UTM/Programming-Techniques main ls
1 'Laborator 2.pdf' 'Laborator 4.pdf' probe README.md web_lab.zip
arsenal 'Laborator 3.pdf' 'Laborator 5.pdf' probe_pregateste.py teste



LUCRAREA A (nucleu). Decoderul universal: arsenal/forensics/decode.py

Obiectiv: un program care primește un șir sau un fișier, recunoaște stratul și îl desface; iar pentru un XOR cu cheie de un octet sau un hash de parolă, îl sparge singur. Pornești de la scheletul de mai jos; funcțiile marcate cu TODO sunt ale tale.

A1. Text și octeți: încălzirea (10 min)

Creează arsenal/forensics/decode.py. Fixează-ți întâi diferența dintre text și octeți, de care atârnă tot.

```
s = "Salut"
b = s.encode()        # bytes
print(b, b[0])        # b"Salut" 83
# TODO: tipareste b.hex() si refa bytes din hex
#       cu bytes.fromhex(...)
```

> **De observat.** b[0] e un număr, nu o literă. Tot ce urmează lucrează pe bytes, nu pe text.

```
import argparse
import base64
import hashlib
import json
import re
from urllib.parse import unquote

#A1
def demo_a1():
    s = "Salut"
    b = s.encode()
    print(b, b[0])
    h = b.hex()
    print(h, bytes.fromhex(h))
```

A2. Recunoști stratul după formă (14 min)

Scrie o funcție care, dat un șir, ghicește ce codare e, uitându-se la caractere. Nu decoda încă, doar clasifică.

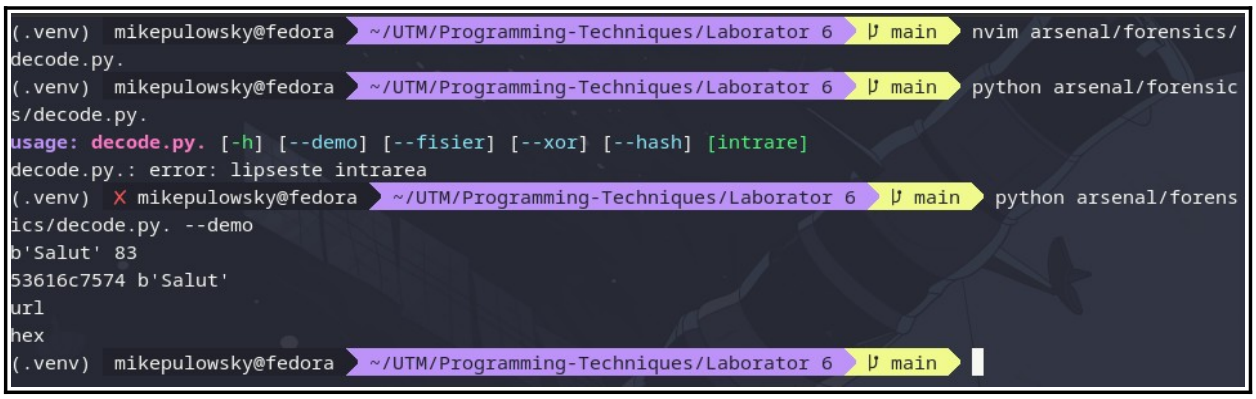
```
import re
def ghici_strat(s):
  # TODO: daca are %xx -> "url"
  # TODO: daca doar 0-9 a-f si lungime para -> "hex"
  # TODO: daca doar A-Za-z0-9+/= -> "base64"
  return "necunoscut"

print(ghici_strat("%2Fetc"))      # url
print(ghici_strat("48656c6c6f"))  # hex
```

> **De observat.** Ordinea contează: verifică întâi „%" (URL), apoi hexul (mai strict), apoi base64 (cel mai larg alfabet).

```
#A2
def demo_a2():
    print(ghici_strat("%2Fetc"))
    print(ghici_strat("48656c6c6f"))

def ghici_strat(s):
    if re.search(r"%[0-9A-Fa-f]{2}", s):
        return "url"
    if re.fullmatch(r"[0-9a-fA-F]+", s) and len(s) % 2 == 0:
        return "hex"
    if re.fullmatch(r"[A-Za-z0-9+/]+={0,2}", s) and len(s) % 4 == 0:
        return "base64"
    return "necunoscut"
```



A3. Desfaci stratul recunoscut (14 min)

Acum chiar decodează, în funcție de ce a zis A2. Întoarce și tipul, și rezultatul.

```
import base64
from urllib.parse import unquote

def desfa(s):
  strat = ghici_strat(s)
  # TODO: pentru fiecare strat, decodeaza
  #   base64 -> base64.b64decode(s)
  #   hex    -> bytes.fromhex(s)
  #   url    -> unquote(s).encode()
  # intoarce (strat, rezultat_bytes)
  return strat, s.encode()
```

```
#A3
def desfa(s):
    strat = ghici_strat(s)
    try:
        if strat == "base64":
            return strat, base64.b64decode(s)
        if strat == "hex":
            return strat, bytes.fromhex(s)
        if strat == "url":
            return strat, unquote(s).encode()
    except ValueError:
        pass
    return "necunoscut", s.encode()
```

A4. Straturi multiple, într-o buclă (14 min)

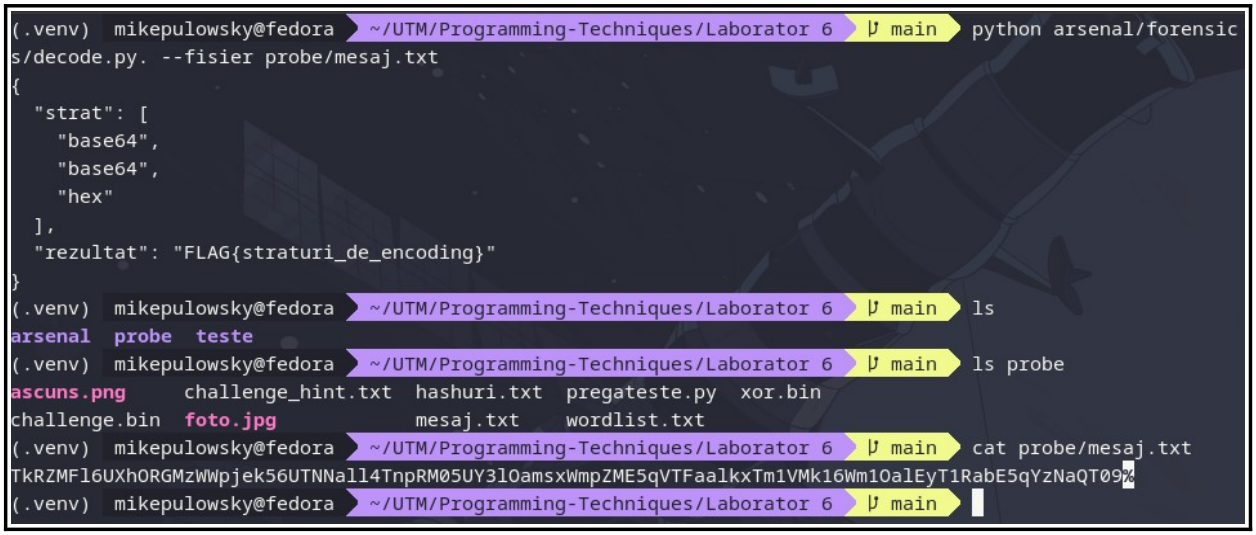
Un rezultat poate fi tot o codare. Decodează în buclă până iese text citibil (sau te oprești după câteva încercări, ca să nu intri în buclă infinită).

```
def citibil(b):
  # TODO: True daca toti octetii sunt 32..126
  return all(32 <= c < 127 for c in b)
```

```
val = "NTM1OTU3NTk="
for _ in range(8):
  strat, b = desfa(val)
  # TODO: daca citibil(b): opreste si tipareste
  #       altfel val = b.decode(); continua
```

```
#A4
def citibil(b):
    return all(32 <= c < 127 for c in b)

def desface_tot(s, max_straturi=8):
    """Desface in bucla. Intoarce (lista_straturi,
rezultat_bytes)."""
    straturi = []
    curent = s.encode() if isinstance(s, str) else s
    for _ in range(max_straturi):
        if not citibil(curent):
            break
        strat, b = desfa(curent.decode())
        if strat == "necunoscut" or not citibil(b):
            break
        straturi.append(strat)
        curent = b
    return straturi, curent
```



A5. Spargi un XOR cu cheie de un octet (16 min)

Citește probe/xor.bin (octeți), încearcă toate cele 256 de chei și păstrează rezultatul care arată a text.

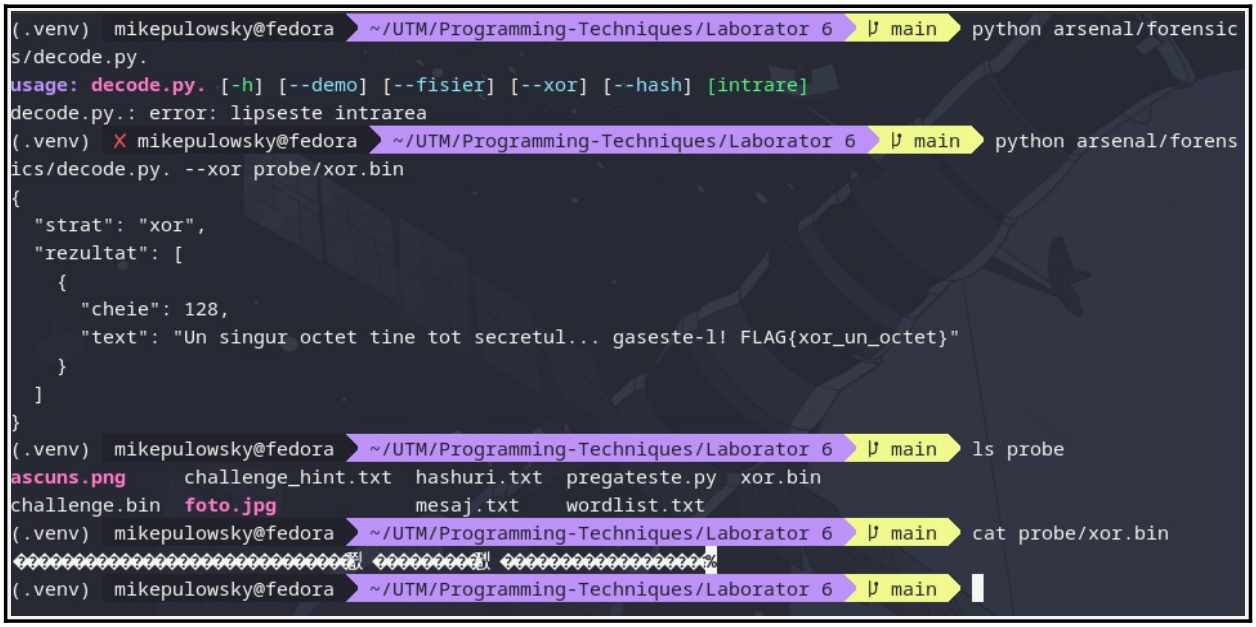
```
date = open("probe/xor.bin", "rb").read()
for k in range(256):
  clar = bytes(b ^ k for b in date)
```

```
    # TODO: daca citibil(clar):
    #     print("cheie:", k, clar.decode())
```

> **De observat.** Operatorul ^ e XOR; b ^ k aplică cheia pe un octet. O cheie de un octet are fix 256 de valori, deci le încerci pe toate.

```
#A5
def scor_text(clar):
    """Ce parte din octeti arata a text (litere, spatiu, {}_)."""
    ok = sum(chr(c).isalpha() or chr(c) in " {}_" for c in clar)
    return ok / len(clar)

def sparge_xor(date, prag=0.8):
    gasite = []
    for k in range(256):
        clar = bytes(b ^ k for b in date)
        if clar and citibil(clar) and scor_text(clar) >= prag:
            gasite.append((k, clar.decode()))
    return sorted(gasite, key=lambda x: -scor_text(x[1].encode()))
```



A6. Spargi un hash prin dicționar (16 min)

Din probe/hashuri.txt ia un hash. Identifică algoritmul după lungime, apoi caută parola într-o wordlist.

```
import hashlib
tinta = open("probe/hashuri.txt").readline().strip()
print(len(tinta))   # 32 md5 / 40 sha1 / 64 sha256
for cuv in open("probe/wordlist.txt", encoding="latin1"):
    cuv = cuv.strip()
    # TODO: daca hashlib.md5(cuv.encode()).hexdigest()
    #       == tinta: print("gasit:", cuv); break
```

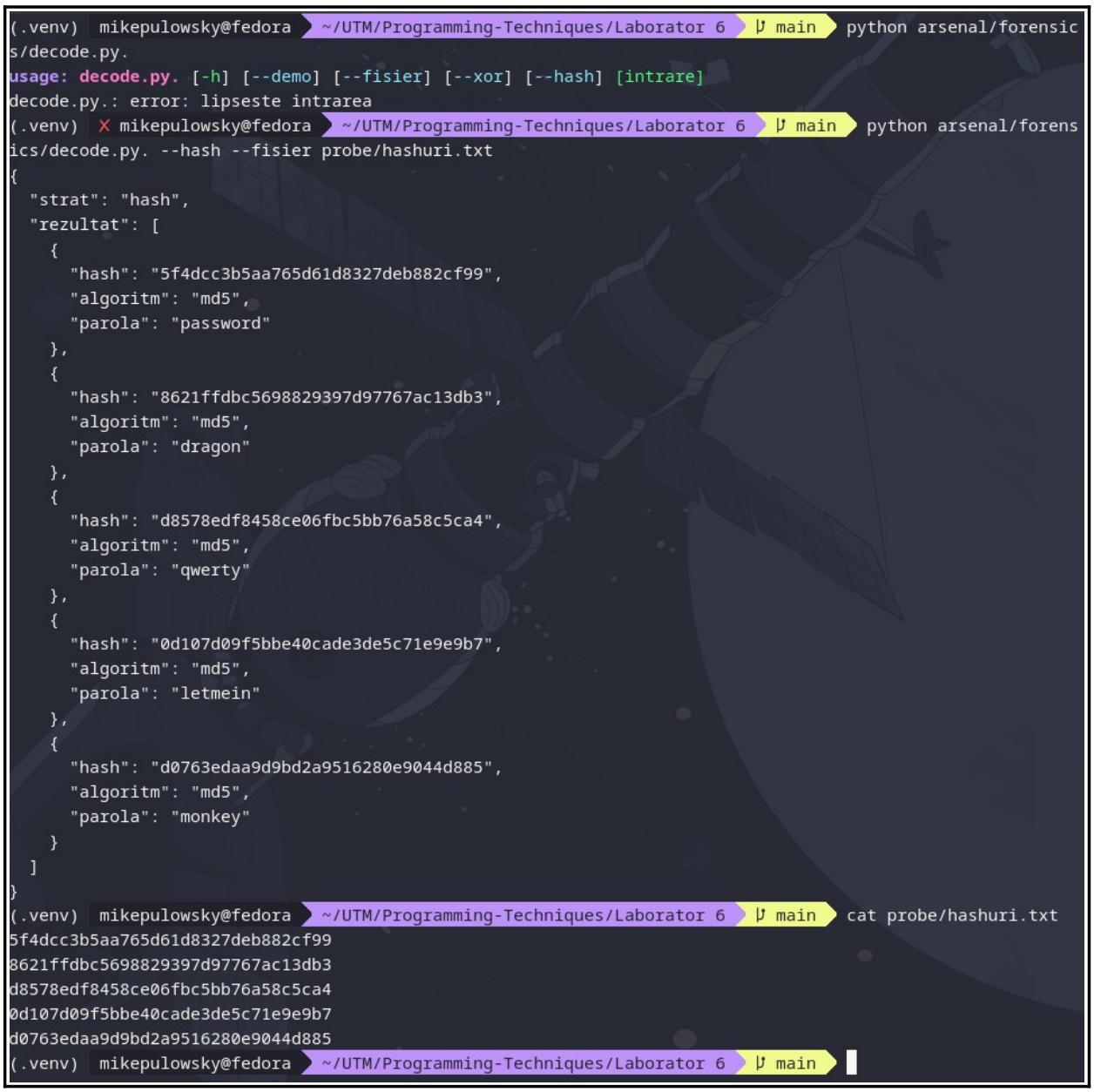
> **De observat.** Nu „decriptezi" hash-ul: hashuiești fiecare cuvânt și compari. Lungimea îți spune ce algoritm să folosești.

```
#A6
```

```
ALGO = {32: "md5", 40: "sha1", 64: "sha256"}

def sparge_hash(tinta, wordlist="probe/wordlist.txt"):
    algo = ALGO.get(len(tinta))
    if not algo:
        return None, None
    with open(wordlist, encoding="latin1") as f:
        for cuv in f:
            cuv = cuv.strip()
            if hashlib.new(algo, cuv.encode()).hexdigest() ==
tinta.lower():
                return algo, cuv
    return algo, None


def hashuri_din_fisier(cale):
    """Accepta 'hash' sau 'nume:hash' pe linie."""
    out = []
    for linie in open(cale):
        for tok in re.split(r"[\s:]+", linie.strip()):
            if re.fullmatch(r"[0-9a-fA-F]+", tok) and len(tok) in
ALGO:
                out.append(tok)
    return out
```



A7. argparse, fișa JSON și un test (16 min)

Leagă totul într-o unealtă comandată din afară, salvează rezultatul ca fișă, și scrie un test.

```
import argparse, json
p = argparse.ArgumentParser()
p.add_argument("intrare")   # sir sau cale
p.add_argument("--fisier", action="store_true")
a = p.parse_args()
# TODO: daca a.fisier, citeste octetii din cale;
#       altfel trateaza intrarea ca sir
# TODO: salveaza {strat, rezultat} in
#       arsenal/forensics/fisa.json
```

Un test scurt care ține decoderul cinstit:

```
# teste/test_decode.py
from arsenal.forensics.decode import desfa
```

```
def test_base64():
    strat, b = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert b == b"Salut"
```

Predai: decode.py, fisa.json, teste/test_decode.py, plus în raport.md: ce ascundea mesajul cu straturi, ce cheie XOR ai găsit și ce parolă ai spart.

```
# A7
def main():
    p = argparse.ArgumentParser(description="Decoder universal")
    p.add_argument("intrare", nargs="?", help="sir sau cale")
    p.add_argument("--demo", action="store_true", help="ruleaza A1 si
A2")
    p.add_argument("--fisier", action="store_true", help="intrarea e
o cale")
    p.add_argument("--xor", action="store_true", help="sparge XOR (1
octet)")
    p.add_argument("--hash", action="store_true", help="sparge
hash(uri)")
    a = p.parse_args()

    if a.demo:
        demo_a1()
        demo_a2()
        return
    if a.intrare is None:
        p.error("lipseste intrarea")

    if a.xor:
        date = open(a.intrare, "rb").read()
        fisa = {"strat": "xor",
                "rezultat": [{"cheie": k, "text": t} for k, t in
sparge_xor(date)]}
    elif a.hash:
        tinte = hashuri_din_fisier(a.intrare) if a.fisier else
[a.intrare]
        rez = []
        for t in tinte:
            algo, parola = sparge_hash(t)
            rez.append({"hash": t, "algoritm": algo, "parola":
parola})
        fisa = {"strat": "hash", "rezultat": rez}
    else:
        if a.fisier:
            s = open(a.intrare,
"rb").read().decode(errors="replace").strip()
```

```
        else:
            s = a.intrare
        straturi, rez = desface_tot(s)
        fisa = {"strat": straturi, "rezultat":
rez.decode(errors="replace")}

    print(json.dumps(fisa, indent=2, ensure_ascii=False))
    with open("arsenal/forensics/fisa.json", "w", encoding="utf-8")
as f:
        json.dump(fisa, f, indent=2, ensure_ascii=False)


if __name__ == "__main__":
    main()
```




```
#A7
from arsenal.forensics.decode import desfa, ghici_strat, desface_tot,
sparge_xor

def test_base64():
```

```
    strat, b = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert b == b"Salut"


def test_hex_si_url():
    assert ghici_strat("48656c6c6f") == "hex"
    assert desfa("48656c6c6f")[1] == b"Hello"
    assert ghici_strat("%2Fetc") == "url"
    assert desfa("%2Fetc")[1] == b"/etc"


def test_straturi_multiple():
    straturi, rez = desface_tot("NTM2MTZjNzU3NA==")
    assert straturi == ["base64", "hex"]
    assert rez == b"Salut"


def test_xor():
    date = bytes(c ^ 42 for c in b"mesaj secret")
    assert (42, "mesaj secret") in sparge_xor(date)
```

```
(.venv)  mikepulowsky@fedora  ~/UTM/Programming-Techniques/Laborator 6  main  pip install pytest
Requirement already satisfied: pytest in /home/mikepulowsky/UTM/Programming-Techniques/.venv/lib64/python3
.14/site-packages (9.1.1)
Requirement already satisfied: iniconfig>=1.0.1 in /home/mikepulowsky/UTM/Programming-Techniques/.venv/lib
64/python3.14/site-packages (from pytest) (2.3.1)
Requirement already satisfied: packaging>=22 in /home/mikepulowsky/UTM/Programming-Techniques/.venv/lib64/
python3.14/site-packages (from pytest) (26.3)
Requirement already satisfied: pluggy<2,>=1.5 in /home/mikepulowsky/UTM/Programming-Techniques/.venv/lib64
/python3.14/site-packages (from pytest) (1.7.0)
Requirement already satisfied: pygments>=2.7.2 in /home/mikepulowsky/UTM/Programming-Techniques/.venv/lib6
4/python3.14/site-packages (from pytest) (2.21.0)
(.venv)  mikepulowsky@fedora  ~/UTM/Programming-Techniques/Laborator 6  main  python -m pytest -q
....                                                                                          [100%]
4 passed in 0.01s
(.venv)  mikepulowsky@fedora  ~/UTM/Programming-Techniques/Laborator 6  main
```

LUCRAREA B (extindere). Forensics pe fișiere și un mesaj ascuns

Obiectiv: citești ce spun fișierele despre ele (semnătură, text, metadate), extragi un fișier ascuns, apoi ascunzi și dezvălui un mesaj prin steganografie LSB. Se lucrează pe fișierele din probe/.

B1. Magic bytes: tipul real, nu extensia (12 min)

Deschide un fișier în mod binar și citește-i primii octeți. Compară-i cu semnăturile cunoscute.

```
def tip_real(cale):
    cap = open(cale, "rb").read(8)
    # TODO: daca cap incepe cu b"%PDF" -> "pdf"
    #       b"\x89PNG" -> "png" ; b"\xff\xd8" -> "jpeg"
    #       b"PK\x03\x04" -> "zip/docx"
    return cap.hex()
```

```
print(tip_real("probe/foto.jpg"))
```

```
import io
import re
import zipfile
from PIL import Image
from PIL.ExifTags import TAGS, GPSTAGS


#B1
SEMNATURI = [
    (b"%PDF", "pdf"),
    (b"\x89PNG", "png"),
    (b"\xff\xd8", "jpeg"),
    (b"PK\x03\x04", "zip/docx"),
]


def tip_real(cale):
    cap = open(cale, "rb").read(8)
    for sig, nume in SEMNATURI:
        if cap.startswith(sig):
            return nume
    return cap.hex()
```

> **De observat. Modul "rb" nu e opțional pentru fișiere binare: fără el, octeții se pot altera.**

B2. strings: textul citibil dintr-un binar (12 min)

Scoate secvențele de caractere imprimabile dintr-un fișier, cu un regex pe octeți.

```
import re
date = open("probe/ascuns.png", "rb").read()
# TODO: for m in re.findall(rb"[ -~]{4,}", date):
#         print(m.decode())
# cauta un indiciu: o cale, un mesaj, un nume de fisier
```

```
#B2
def strings(cale, minim=4):
    date = open(cale, "rb").read()
    return [m.decode() for m in re.findall(rb"[ -~]{%d,}" % minim,
date)]
```

B3. Metadate EXIF și GPS din poză (16 min)

Citește blocul EXIF din probe/foto.jpg și scoate coordonatele GPS. Convertește-le în grade zecimale.

```
from PIL import Image
from PIL.ExifTags import TAGS
```

```
exif = Image.open("probe/foto.jpg")._getexif() or {}
for id, val in exif.items():
   print(TAGS.get(id, id), "=", val)
# TODO: ia blocul GPS (tag 34853) si tipareste
#       latitudinea si longitudinea
```

> **De observat.** Coordonatele vin ca (grade, minute, secunde) plus emisfera. Le poți pune pe o hartă: de aici valoarea lor pentru OSINT, și motivul să ștergi EXIF înainte de a publica.

```
#B3
def _grade(dms, ref):
    d, m, s = (float(x) for x in dms)
    val = d + m / 60 + s / 3600
    return -val if ref in ("S", "W") else val


def exif_gps(cale):
    exif = Image.open(cale).getexif()
    info = {}
    for tid, val in list(exif.items()) +
list(exif.get_ifd(0x8769).items()):
        info[TAGS.get(tid, tid)] = val
    gps = {GPSTAGS.get(k, k): v for k, v in
exif.get_ifd(34853).items()}
    coord = None
    if "GPSLatitude" in gps and "GPSLongitude" in gps:
        lat = _grade(gps["GPSLatitude"], gps.get("GPSLatitudeRef",
"N"))
        lon = _grade(gps["GPSLongitude"], gps.get("GPSLongitudeRef",
"E"))
        coord = (lat, lon)
    return info, gps, coord
```

B4. File carving: scoți fișierul ascuns (14 min)

probe/ascuns.png are o arhivă ZIP lipită la coadă. Găsește-i semnătura și salveaz-o separat.

```
date = open("probe/ascuns.png", "rb").read()
i = date.find(b"PK\x03\x04")
# TODO: daca i != -1:
#       open("gasit.zip","wb").write(date[i:])
#       print("arhiva extrasa de la octetul", i)
# apoi dezarhiveaza si vezi ce era inauntru
```

```
def carve_zip(cale, out="gasit.zip"):
    date = open(cale, "rb").read()
    i = date.find(b"PK\x03\x04")
    if i == -1:
```

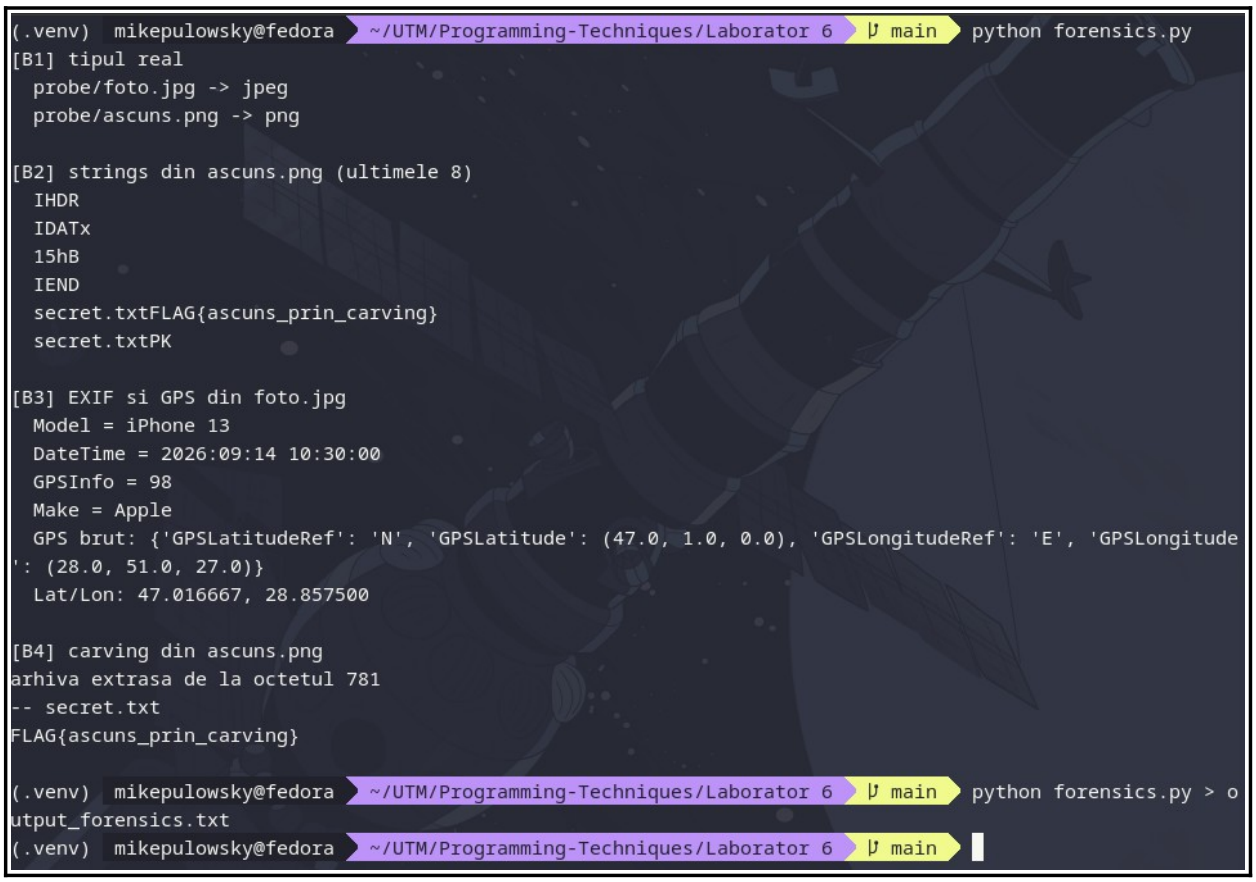
```
        return None
    open(out, "wb").write(date[i:])
    print("arhiva extrasa de la octetul", i)
    with zipfile.ZipFile(io.BytesIO(date[i:])) as z:
        for nume in z.namelist():
            print("--", nume)
            print(z.read(nume).decode(errors="replace"))
    return i


if __name__ == "__main__":
    print("[B1] tipul real")
    for f in ("probe/foto.jpg", "probe/ascuns.png"):
        print(" ", f, "->", tip_real(f))

    print("\n[B2] strings din ascuns.png (ultimele 8)")
    for s in strings("probe/ascuns.png")[-8:]:
        print(" ", s)

    print("\n[B3] EXIF si GPS din foto.jpg")
    info, gps, coord = exif_gps("probe/foto.jpg")
    for k, v in info.items():
        print(f"  {k} = {v}")
    print("  GPS brut:", gps)
    if coord:
        print(f"  Lat/Lon: {coord[0]:.6f}, {coord[1]:.6f}")

    print("\n[B4] carving din ascuns.png")
    carve_zip("probe/ascuns.png")
```



B5. Ascunzi un mesaj prin LSB (16 min)

Scrie un mesaj în ultimii biți ai pixelilor dintr-o poză gazdă. Rezultatul arată identic cu originalul.

```
from PIL import Image
img = Image.open("probe/foto.jpg").convert("RGB")
px = list(img.getdata())
mesaj = b"FLAG{ascuns_in_pixeli}"
biti = "".join(f"{x:08b}" for x in mesaj)
# TODO: pentru fiecare bit, ia un canal si pune
#       canal = (canal & ~1) | int(bit)
# TODO: img.putdata(px_nou); img.save("stego.png")
```

```
import sys
from PIL import Image

def pixeli(img):
    return getattr(img, "get_flattened_data", img.getdata)()


#B5
def ascunde(gazda, mesaj, out="stego.png"):
    img = Image.open(gazda).convert("RGB")
    canale = [c for px in pixeli(img) for c in px]
    biti = "".join(f"{x:08b}" for x in mesaj + b"\x00")
```

```
    if len(biti) > len(canale):
        raise ValueError("mesaj prea lung pentru imagine")
    for i, bit in enumerate(biti):
        canale[i] = (canale[i] & ~1) | int(bit)
    img.putdata([tuple(canale[i:i + 3]) for i in range(0,
len(canale), 3)])
    img.save(out)
    return out
```

> **De observat.** Salvează în .png (fără pierdere); un .jpg recompresează imaginea și distruge biții ascunși.

B6. Dezvălui mesajul înapoi (14 min)

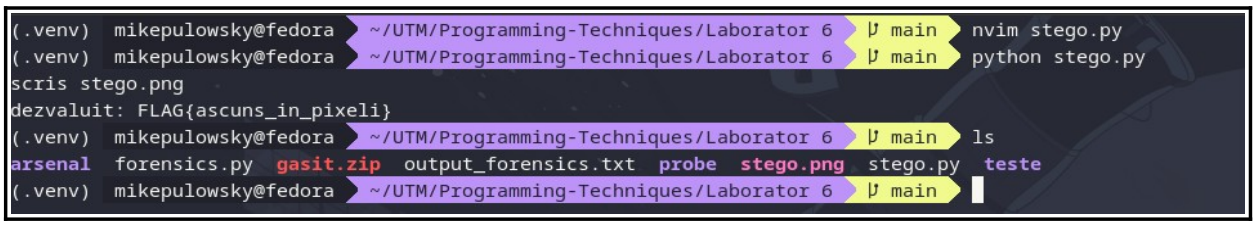
Citește ultimii biți ai pixelilor din stego.png, regrupează-i câte 8 și reconstituie mesajul.

```
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
  # TODO: aduna ultimii biti (r&1, g&1, b&1)
  pass
# TODO: grupeaza cate 8 biti -> octeti -> text
#     opreste-te la primul octet 0 (final de mesaj)
```

> **De observat.** Dezvăluirea e exact inversul ascunderii: aceiași pași, citiți invers. Când apare text citibil, ai găsit mesajul.

```
#B6
def dezvaluie(cale="stego.png"):
    canale = [c for px in pixeli(Image.open(cale).convert("RGB")) for
c in px]
    biti = "".join(str(c & 1) for c in canale)
    octeti = bytearray()
    for i in range(0, len(biti) - 7, 8):
        o = int(biti[i:i + 8], 2)
        if o == 0:
            break
        octeti.append(o)
    return bytes(octeti).decode(errors="replace")


if __name__ == "__main__":
    mesaj = sys.argv[1].encode() if len(sys.argv) > 1 else
b"FLAG{ascuns_in_pixeli}"
    ascunde("probe/foto.jpg", mesaj)
    print("scris stego.png")
    print("dezvaluit:", dezvaluie("stego.png"))
```



PENTRU VÂRFUL SĂLII (opțional)

Dacă termini devreme, în probe/ găsești challenge.bin: un fișier care combină tot ce ai făcut azi (un strat de encoding peste un XOR, cu un hash de verificat). Despică-l singur, fără schelet, și pune pașii în raport. Este genul de problemă pe care o vei întâlni pe pwn.college și pe challenge-urile de forensics.

Predai: scripturile de la B1-B6, fișierul stego.png, și în raport.md: tipul real al fișierelor, ce ai găsit în metadate (inclusiv GPS), ce era în arhiva ascunsă, și mesajul LSB dezvăluit.

Ce predai și cum se notează

Livrabil	Ce conține
arsenal/forensics/decode.py	Decoderul: recunoaștere, desfacere, straturi, XOR brute, hash (A2-A7).
arsenal/forensics/fisa.json	Fișa unui mesaj decodat, salvată de unealtă (A7).
teste/test_decode.py	Testul care trece (A7).
forensics.py (sau similar)	Scriptul cu B1-B4: magic bytes, strings, EXIF/GPS, carving.
stego.png + scriptul LSB	Poza cu mesaj ascuns și codul care ascunde/dezvăluie (B5-B6).
raport.md (secțiune nouă)	Ce ai decodat, ce cheie/parolă ai spart, metadate GPS, arhiva ascunsă, mesajul LSB.

Cum predai (obligatoriu, prin git): urci TOT codul în depozitul tău git tehnici-prog-<nume> (cadrul didactic este colaborator) și dai commit + push la finalul lucrării. Asistentul îl descarcă de acolo (git pull) și îl rulează. Nu se trimite nimic pe e-mail și nicio captură de ecran.

```
git add .
git commit -m "Lab 6: decoder universal + forensics si stego"
git push
```

Nucleul (Lucrarea A) se face pe viu, în laborator. Extinderea (Lucrarea B) se poate continua acasă: fișierele din probe/ sunt generate de tine, deci nu depinzi de rețeaua laboratorului. Se notează codul care rulează și output-ul, nu capturi de ecran.